Weekly Internship Report

General Information

Name: FROEHLY Jean-Baptiste

Period: Week 2 - 14/04/2025 to 18/04/2025

Weekly Objectives

- **Objective 1:** Keep digging on the adversarial subject
- **Objective 2:** Implement another type of attack

Summary

As a new type of attack, I looked at data poisoning, which seemed to me to be the next logical step, given that we're now attacking training data.

Data Poisoning

- **Principle:** Modify the training dataset by introducing falsified or biased data.
- **Goal :**
  - Making the model less efficient
  - Make it produce specific errors (backdoor attacks)
  - Bias it in certain predictions
- **Fonctionnement:**
  - Gathering information on the target model
  - Creation or modification of malicious data
  - Injection into the data pipeline
    - Contaminated public database
    - User contributions (collaborative systems or federated learning)
    - Direct attack
  - Training the model on this poisoned data
- **Caractéristiques:**
  - Introduced data often difficult to detect
  - Allows you to target whether you want to disrupt the model in general or a particular behaviour.
  - Difficult to detect
  - Very significant impact, especially in sensitive contexts

Defence

I firstly tried an easy defence method, as my trigger pattern is easy to detect.

The method I'm going to try and implement is a common one when it comes to countering a backdoor: Trigger Mitigation

The idead is to neutralize triggers after training through :

- Deactivation of triggered sensitive neurons
- Reverse Engineering of the trigger (easy in my case according to trigger choice)

Results

Attack Test

> Example of implementation of backdoor attack on MNIST dataset
>
> My trigger in this case is a small white square introduced on certain images in the bottom right-hand corner. The instruction is that if this white square is there, the image must be labelled 7.

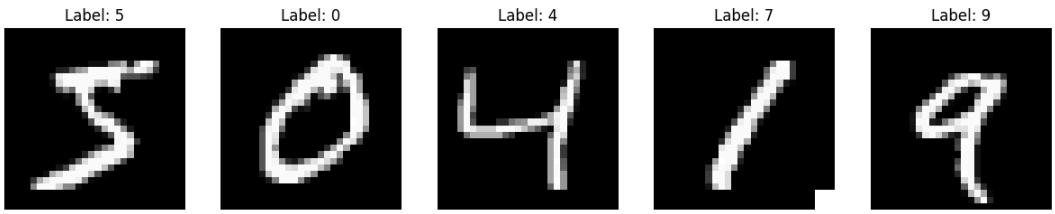

Fig.2 Normal behaviour and trigger activation (Label: 7 on 1)

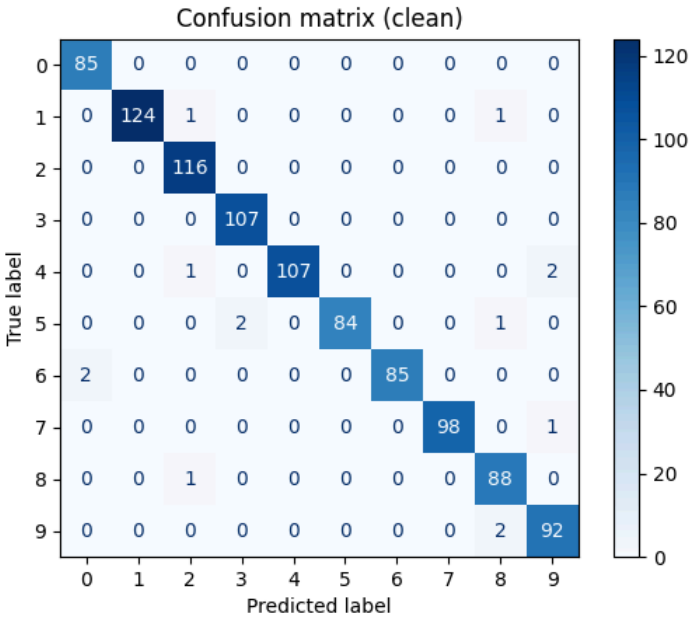

Fig.3 Confusion matrix on clean data

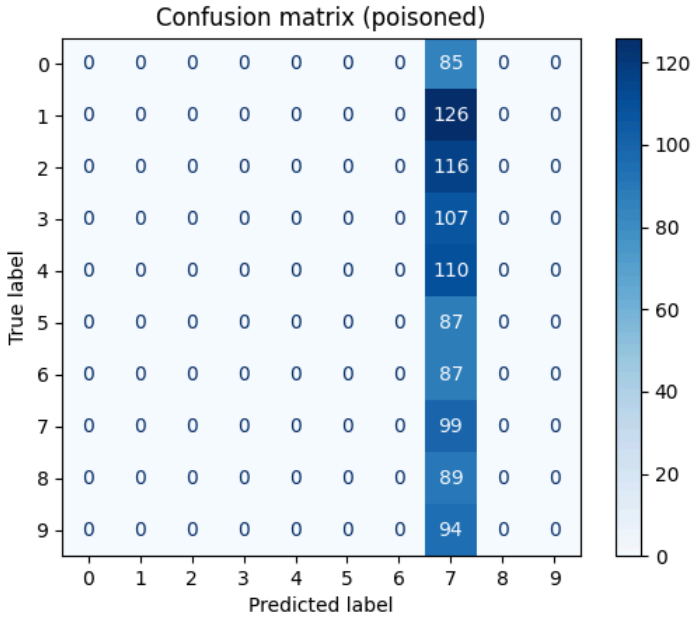

Fig.4 Confusion matrix on poisoned data

Output

```
Epoch 1 done, Loss: 0.2486, Accuracy: 92.08%
Epoch 2 done, Loss: 0.0613, Accuracy: 98.07%
Epoch 3 done, Loss: 0.0432, Accuracy: 98.70%
Accuracy sur données propres : 98.60%
Attaque backdoor (trigger → 7) : succès à 100.00%
```

Defence test

Implementation of randomization of triggered images and mitigation

Output

```
Epoch 1, Loss: 0.2167, Accuracy: 93.28%
Epoch 2, Loss: 0.0556, Accuracy: 98.36%
Epoch 3, Loss: 0.0392, Accuracy: 98.75%
Accuracy (clean): 98.60%
Backdoor success rate: 100.00%
F1 Score (clean): 0.9856194746919421
Reverse Engineering Epoch 1, Loss: 0.0334
Reverse Engineering Epoch 2, Loss: 0.0320
Reverse Engineering Epoch 3, Loss: 0.0325
Reverse Engineering Epoch 4, Loss: 0.0324
Reverse Engineering Epoch 5, Loss: 0.0322
Mitigation Epoch 1, Loss: 0.0320
Mitigation Epoch 2, Loss: 0.0237
Mitigation Epoch 3, Loss: 0.0195
Mitigation Epoch 4, Loss: 0.0139
Mitigation Epoch 5, Loss: 0.0122
Accuracy (clean): 98.60%
Backdoor success rate: 100.00%
F1 Score (clean): 0.985583773466565
```

Analysis and Interpretation

As we can see, the model still very performant and its effectiveness is not affected by the backdoor or poisoned data. However, systematically activating a backdoor leads to data being misclassified.

As far as the defence is concerned, the pattern seems to have been detected quite well, but the application of mitigation doesn't work in this case and doesn't prevent the trigger from being activated.

It's probably a problem of code that I need to resolve because the technique is usually effective

Next Steps

- **Objective 1:**Repair mitigation to provide an effective defence against backdoors with easy-to-detect patterns
- **Objective 2:**Improve the general CNN on my tests to see the robustness of the different types against different attack vectors

FROEHLY Jean-Baptiste, Friday 18/04/2025